

The Cold-Start Tax: Warm-Up, Re-Warm, and Residency Policy for Duty-Cycled Edge DNN Inference

Manu Nicholas Jacob

Abstract—Edge inference benchmarks, and the deployment guidance drawn from them, almost universally report *steady-state* latency and energy: the numbers a model reaches after it is loaded and warmed. Yet a large class of Internet-of-Things devices does not run inference in a hot loop. They *duty-cycle*—wake on an event or a timer, run a handful of inferences, and return to a low-power state—so that the transient at every wake, not the steady state, dominates their user-visible latency and their energy budget. We call this transient the *cold-start tax* and present the first systematic characterization of it on a representative edge platform, the Raspberry Pi 5 (ARM Cortex-A76), across twelve models spanning compact CNNs, dense CNNs, statically-quantized INT8 CNNs, and a vision transformer. We measure a cold-start tax of $5\text{--}23\times$ the steady-state latency and decompose it into disk (flash) weight loading, graph construction and optimization, arena allocation, and first-inference compute. The tax is overwhelmingly a *weight-loading* cost (55–81 % of the wake); the first inference itself runs at steady-state speed, so there is essentially no compute warm-up to amortize. Using the Pi 5’s on-board PMIC telemetry we show a single cold wake costs the energy of 4–19 steady-state inferences (up to 35 J for a vision transformer), and that on a memory-constrained (2 GB) device the tax *re-triggers* whenever idle-interval memory pressure evicts the model’s pages—an *eviction cliff* we map directly. Because the wake is I/O-bound, the on-demand DVFS governor’s clock ramp adds only a small tax, and static INT8 shrinks the absolute tax but not its ratio. We turn these measurements into a method: GD-TAX, a tax-aware model-residency cache for multi-model edge deployments that keeps warm the sessions whose eviction would cost the most energy. Evaluated on measured constants over realistic request traces, GD-TAX never underperforms LRU or LFU and cuts energy per request by up to 15% at a tight RAM budget, matching an offline Belady reference. We argue that duty-cycled edge inference should be reported and optimized in terms of the cold wake, not the steady state.

Index Terms—Edge AI, deep neural network inference, cold start, duty cycling, page cache, energy efficiency, model serving, Internet of Things, ARM Cortex-A76, Raspberry Pi.

I. INTRODUCTION

THE dominant way to report the cost of running a deep neural network (DNN) on an edge device is to load the model once, run a warm-up loop, and then report the median or tail of a long run of inferences. This *steady-state* convention underlies public leaderboards, vendor datasheets, and most research characterizations [1], [2]. It is the right convention for a server that keeps a model resident and serves a continuous

stream of requests. It is the wrong convention for the way an enormous fraction of IoT endpoints actually operate.

A battery- or energy-harvesting-powered sensor, a wildlife camera, a wearable, or an intermittently-connected industrial monitor does not hold a model hot. To conserve energy it *duty-cycles*: it sleeps, wakes on a passive-infrared trigger or a timer, classifies a few frames, and sleeps again. For such a device the relevant question is not “how fast is the hundredth inference?” but “how long, and how much energy, from wake to first result?” That first-result cost includes work the steady-state number hides entirely: the model’s weights must be brought from flash into memory, the inference runtime must parse the graph, apply its optimizing rewrites, and allocate its memory arenas, and only then can the first inference run. We call the gap between this cold wake and the steady state the *cold-start tax*.

Cold starts are studied intensively in one adjacent setting—serverless / Function-as-a-Service platforms, where container and language-runtime initialization dominate invocation latency [3]–[5]. That literature lives in the data center, measures container and interpreter startup rather than model weight movement, and optimizes a keep-alive policy against cloud billing. To our knowledge the cold-start transient of *on-device* DNN inference on a resource-constrained edge SBC—its magnitude, its physical decomposition, its energy in Joules, and its interaction with a small page cache—has not been characterized. This paper does so, and turns the characterization into an actionable residency policy.

Contributions.

- 1) **A measurement of the tax (§IV).** Across twelve models on a Raspberry Pi 5 we find a cold-start tax of $5\text{--}23\times$ the steady-state latency, and decompose it into weight loading, graph build/optimize, arena allocation, and cold compute. The tax is dominated by weight loading from flash (55–81% of the wake); the first inference runs at steady-state speed, so—contrary to the intuition behind “warm-up loops”—there is almost no compute transient to amortize (§IV-A).
- 2) **The eviction cliff (§V).** On a 2 GB device the tax is not paid once but *re-paid* whenever idle-interval memory pressure evicts the model’s pages from the page cache. We map next-wake latency against page-cache pressure and expose a sharp cliff that governs how often a duty-cycled device actually pays full price.
- 3) **The energy of a wake (§VI).** Using the Pi 5’s PMIC power telemetry we show a single cold wake costs the

M. N. Jacob is with the Department of Electrical and Computer Engineering. E-mail: manunicholasjacob@gmail.com.

Artifact (code, raw measurements, and figures) available at the project repository.

energy of 4–19 steady-state inferences, that the wake is I/O-bound so the on-demand DVFS governor’s ramp adds only a small tax (§VI-A), and that static INT8 quantization shrinks the absolute tax but leaves its ratio essentially unchanged.

- 4) **A tax-aware residency policy (§VIII).** We cast multi-model edge serving under a RAM budget as a cost-aware caching problem and propose GD-TAX, a Greedy-Dual policy keyed on each model’s measured cold-wake energy penalty. On measured constants over realistic traces it never underperforms LRU/LFU and cuts energy per request by up to 15% at tight RAM budgets, matching an offline Belady reference.

All measurements are taken on commodity hardware with an open, scripted artifact.

II. RELATED WORK

Edge DNN benchmarking. Systematic characterizations of DNN inference on mobile and edge hardware [2], [6], [7] and standard suites such as MLPerf [1] report steady-state latency, throughput, and (increasingly) energy. They deliberately exclude load and warm-up to isolate the compute kernel. Our companion work on the same platform characterizes the steady-state *memory wall* [8] and the steady-state *energy/DVFS* trade-off [9]. The present paper is orthogonal: it measures precisely the transient those studies exclude, which is the part that dominates a duty-cycled deployment.

Serverless cold starts. FaaS platforms suffer cold starts dominated by sandbox and runtime initialization; measurement studies [3], [4] and keep-alive policies such as FaasCache’s greedy-dual eviction [5] address them in the data center. We borrow the greedy-dual framing [10] but target a fundamentally different regime: on-device inference on a single memory-constrained SBC, where the dominant cost is DNN weight movement from flash into a small page cache, the currency is measured Joules rather than cloud billing, and the resource under contention is 2 GB of physical RAM.

Inference serving and predictability. Systems such as Clockwork [11] and INFaaS [12] manage model loading and placement for predictable serving on servers and GPUs. Their mechanisms (proactive loading, model-less selection) assume abundant memory and target multi-tenant clusters; we study the single-device edge limit and the physical eviction behavior of the OS page cache that those systems abstract away.

Model compression and quantization. Pruning and quantization [13], [14] reduce model size and steady-state cost. We show quantization’s effect on the *cold-start* tax is more subtle: a smaller INT8 file reduces the absolute load time but, because steady-state latency falls in step, the tax *ratio* is preserved.

III. EXPERIMENTAL SETUP

Platform. All experiments run on a Raspberry Pi 5 (Broadcom BCM2712, quad-core ARM Cortex-A76 up to 2.4 GHz, 2 GB LPDDR4X, 32 GB class-10 microSD, Debian 12, Linux 6.12, 64-bit). We use ONNX Runtime [15] 1.24 with the CPU execution provider and, unless stated otherwise, four intra-op threads and its default (all) graph-optimization level.

Models. We use twelve pre-trained ImageNet models spanning the design space: compact CNNs (SqueezeNet 1.1 [16], ShuffleNet V2 [17], MobileNetV2 [18]), dense CNNs (ResNet-18/50 [19], GoogLeNet [20], DenseNet [21], EfficientNet-Lite4 [22]), three statically-quantized INT8 CNNs, and a vision transformer (ViT-Base [23]). Model files range from 1.3 MB to 347 MB.

Cold-start control. A wake is made fully cold by flushing the kernel page cache (`sync; echo 3 > /proc/sys/vm/drop_caches`) immediately before session creation, forcing the model file to be re-read from flash. To isolate the disk-read component we compare against a “warm-file” condition in which the model file is first read into the page cache, so session creation performs no disk I/O. The graph-optimization component is isolated by comparing the default optimization level against a disabled-optimization build. Page faults are accounted via `getrusage` (major faults = disk-backed).

Timing, energy, and clock control. Latencies use `perf_counter`; each data point is the median of 5–7 cold trials with the interquartile range reported. Energy is integrated from the Pi 5’s on-board PMIC (`vcgencmd pmic_read_adc`, per-rail $V \times I$ at ~ 50 Hz), isolating the VDD_CORE (CPU) and DDR rails as in our energy study [9]. Except in the DVFS experiment (§VI-A), the CPU clock is pinned to 2.4 GHz via the `userspace` governor so that the cold-start measurement is not confounded by governor ramp-up.

IV. THE COLD-START TAX AND ITS DECOMPOSITION

Table I reports, for every model, the steady-state latency, the cold-wake latency (session creation from a flushed cache plus the first inference), and their ratio—the cold-start tax. The tax ranges from $4.8\times$ (SqueezeNet-INT8) to $22.5\times$ (ShuffleNet-V2), i.e., the very first result after a wake takes 5–23 times as long as a steady-state inference. Absolute cold-wake latency spans 59 ms (SqueezeNet-INT8) to 1.41 s (ResNet-50). For a device that wakes, classifies one frame, and sleeps, this transient—not the steady state—is the entire user-visible latency.

Figure 2 decomposes the cold wake into four components isolated as described in §III: loading weights from flash, graph build and arena allocation, graph optimization, and cold compute (the first inference above steady state). The decomposition yields the paper’s central mechanical finding: *the cold-start tax is overwhelmingly a weight-loading cost*. Flash read accounts for 55 % to 81 % of the cold wake for the FP32 CNNs (and 48–81% across all models); the effective sequential read bandwidth from the microSD card is only ~ 94 MB/s (ResNet-50: 102.6 MB in 1.09 s), so larger models pay proportionally. Graph optimization is a secondary but non-trivial cost for the deeper graphs (e.g., 110 ms for ResNet-50, 83 ms for ResNet-50-INT8, whose quantize/dequantize nodes are numerous).

A. There Is No Compute Warm-Up

A widespread practice is to run a “warm-up loop” of several inferences before measuring, on the assumption that the first

TABLE I

COLD-START CHARACTERIZATION ON RASPBERRY PI 5 (CORTEX-A76, 2.4 GHz PINNED). LATENCIES ARE MEDIAN OF 5–7 COLD TRIALS. “Disk%” IS THE WEIGHT-LOADING FRACTION OF THE COLD WAKE. ENERGY FROM ON-BOARD PMIC; “= INF.” IS THE COLD-WAKE ENERGY EXPRESSED IN STEADY-STATE INFERENCE.

Model	File (MB)	Steady (ms)	Cold wake (ms)	Tax ($\times$)	Disk%	E_{cold} (J)	= inf.
SqueezeNet-1.1	4.96	12.9	83.7	6.5	73	0.34	4
ShuffleNet-V2	9.22	6.7	150.2	22.5	72	0.85	19
MobileNetV2	13.96	16.7	203.6	12.2	77	1.21	11
GoogLeNet	28.02	51.6	397.5	7.7	77	2.39	7
DenseNet-121	32.73	123.8	635.2	5.1	55	3.82	5
ResNet-18	46.82	46.9	661.7	14.1	75	4.01	12
EfficientNet-Lite4	51.95	60.0	687.9	11.5	81	3.95	10
ResNet-50	102.58	106.6	1410.8	13.2	77	8.80	12
ViT-Base/16	346.53	377	4620	12.3	78	35.2	12
SqueezeNet-1.1-INT8	1.30	12.3	59.1	4.8	48	0.31	4
MobileNetV2-INT8	3.60	7.3	96.7	13.2	55	0.47	11
ResNet-50-INT8	25.75	32.7	432.8	13.2	63	2.98	13

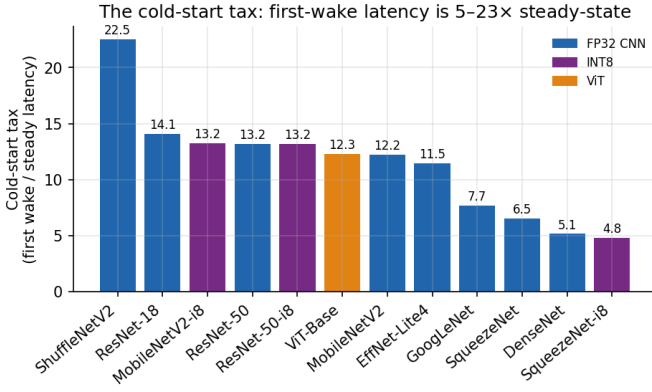


Fig. 1. The cold-start tax: first-wake latency is 5–23 $\times$ the steady-state latency that benchmarks report. INT8 quantization lowers absolute latency but not the tax ratio.

inferences are slow until caches, branch predictors, and lazily allocated buffers settle. Figure 3 shows this assumption does not hold on this platform: once the session exists, the first inference already runs at steady-state speed. Across all models the first-inference latency is within 0% to 11% of steady state (median 1.00), and the number of inferences needed to reach the $\pm 5\%$ steady-state band is 0–1. The cold-start tax therefore lives *entirely* in session construction (loading and preparing the model), not in the execution of the model. Warm-up loops amortize a cost that, on this hardware, does not exist; what must be amortized is the one-time session build.

B. Quantization Shrinks the Tax but Not Its Ratio

Static INT8 quantization is usually adopted for its steady-state speedup. On the cold wake its effect is two-sided. Because an INT8 file is smaller, its absolute cold wake falls sharply (ResNet-50: 1.41 s $\rightarrow$ 0.43 s for ResNet-50-INT8; MobileNetV2: 204 ms $\rightarrow$ 97 ms). But because steady-state latency falls in step, the tax *ratio* is essentially unchanged (ResNet-50: 13.2 $\times$ in both precisions; MobileNetV2: 12.2 $\times$ $\rightarrow$ 13.2 $\times$).

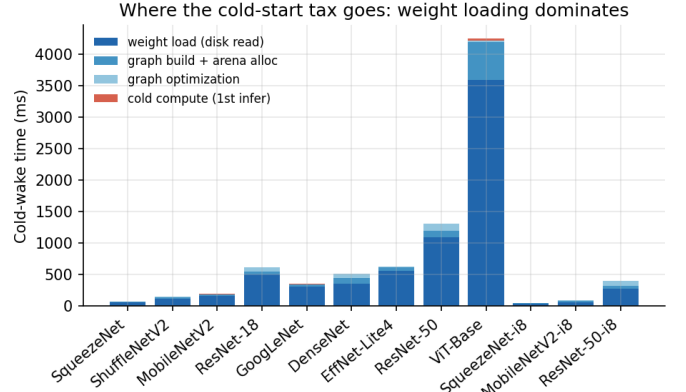


Fig. 2. Decomposition of the cold wake. Loading weights from flash dominates (55–81% for the FP32 CNNs); cold compute is negligible.

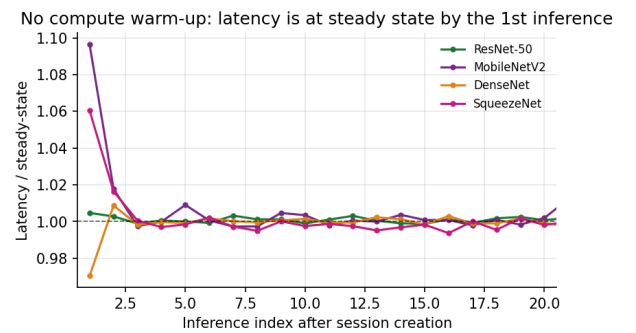


Fig. 3. No compute warm-up: latency is at steady state by the 1st inference. The transient benchmarks guard against with warm-up loops is absent; the real cost is the one-time session build (Fig. 2).

Quantization buys a cheaper wake in absolute terms without making the device any less dominated by its cold start.

C. Storage Is the Cause, Not Model Size

Cold-wake latency correlates with model size, but size is a proxy: the mechanism is bytes moved from flash. To

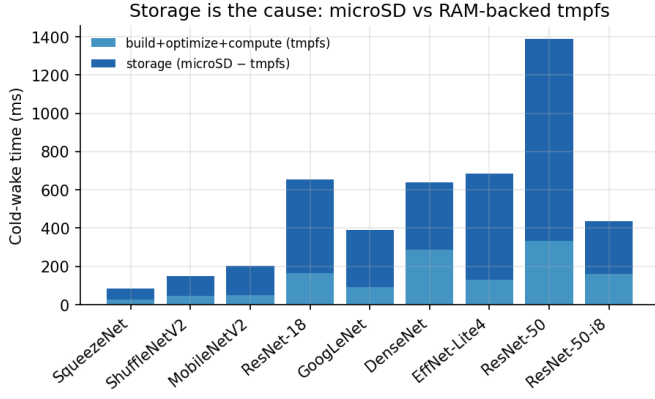


Fig. 4. Storage sensitivity. Loading from RAM-backed tmpfs versus microSD isolates the weight-loading term; the gap (storage) dominates the cold wake for larger models, confirming the tax is flash-bandwidth bound.

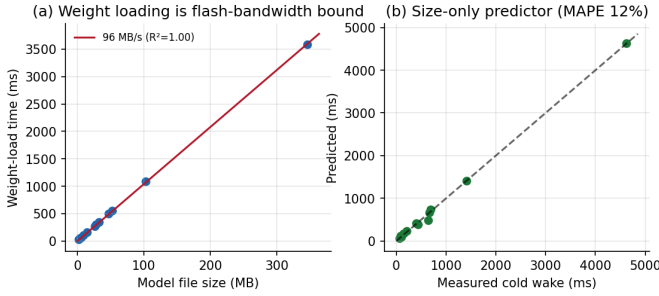


Fig. 5. (a) Weight-load time is linear in file size (microSD ≈ 96 MB/s). (b) A size-only predictor estimates the cold wake to within 12% MAPE.

establish causality we load each model from RAM-backed tmpfs (`/dev/shm`, effectively unbounded storage bandwidth) and compare against the microSD card (Fig. 4). The tmpfs wake removes the storage term, leaving only graph build, optimization, and first compute; the microSD-minus-tmpfs gap is the storage contribution and accounts for 55–81% of the cold wake for the larger models. This confirms the tax is a storage-bandwidth cost and predicts that a faster medium (eMMC, NVMe) would shrink it toward the build/optimize floor rather than eliminate it.

D. A Size-Only Predictor of the Tax

Because loading dominates and is bandwidth-bound, the cold wake is predictable from a single deploy-time quantity: the model’s file size. Fitting the weight-load time against file size gives a near-perfect straight line—an effective microSD read bandwidth of 96 MB/s ($R^2 = 1.00$)—and a size-only model of the whole cold wake, $\hat{t}_{\text{cold}} = 13.2 \text{ ms/MB} \cdot \text{file}_{\text{MB}} + 53 \text{ ms}$, predicts measured cold-wake latency across all twelve models with $R^2 = 0.998$ and a mean absolute percentage error of 12% (Fig. 5). A deployer can therefore estimate the wake penalty of a candidate model, on this class of hardware, from its file size alone—no profiling run required.

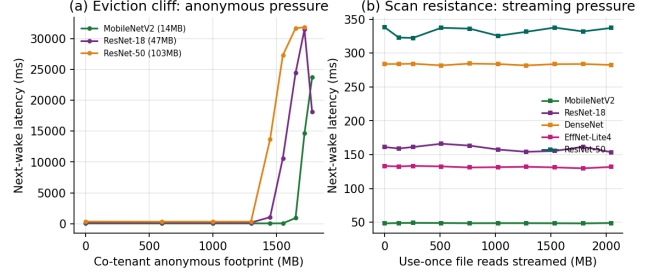


Fig. 6. The eviction cliff. Anonymous co-tenant memory pressure during the idle interval evicts the model’s pages once free memory drops below its footprint, re-triggering the full cold-start tax on the next wake. (Use-once streaming pressure, by contrast, leaves the model warm.)

V. THE EVICTION CLIFF

The tax in Table I is paid on a fully cold cache. Whether a duty-cycled device actually pays it on each wake depends on whether the model’s pages survive the idle interval in the page cache. We find the answer is governed by *what kind* of memory pressure the idle interval brings.

Use-once file I/O does not evict a recently-used model. When we stream up to 2 GB of unrelated file reads through the page cache after using a model, the model’s next wake stays warm: session-create latency remains at its warm value for every model tested, across the whole sweep (e.g., ResNet-50 session-create stays ~ 215 ms, far below its 1.09 s cold value; ResNet-18 stays ~ 110 ms). This is Linux’s active/inactive-list scan resistance: pages touched just before the idle interval sit on the active list and are protected against a use-once streaming scan. For duty cycles whose idle work is I/O-bound (logging, buffering), the model is largely safe.

A co-tenant’s working set does evict it—sharply. Anonymous memory demand is different: to satisfy it the kernel must reclaim file-backed pages, including the model’s. Figure 6 sweeps a co-tenant’s anonymous footprint on the 2 GB device and exposes a sharp *eviction cliff*: below the cliff the wake is warm, and once the footprint pushes available memory below the model’s resident size the next wake jumps to its full cold value—the tax of Table I is re-paid in full. The cliff location sits where a co-tenant drives free memory below the model’s working set: ResNet-50 stays warm (~ 344 ms) until the co-tenant reaches ~ 1.4 GB (leaving < 350 MB free), beyond which session creation thrashes swap and the wake explodes to over 30 s—a $> 80\times$ jump. Every model tested shows the same cliff. On a memory-constrained multi-tenant edge device, then, the cold-start tax is not a one-time startup cost but a recurring one, paid whenever a co-resident workload grows—which directly motivates the residency policy of §VIII.

VI. THE ENERGY OF A WAKE

Latency is only half the cost; for battery- and harvesting-powered devices the energy of a wake sets the duty-cycle budget. Using the Pi 5’s PMIC telemetry we measure the energy of a cold wake, a warm wake, and a steady-state inference (Fig. 7, Table I). A single cold wake costs the energy of 4–19 steady-state inferences. In absolute terms a cold

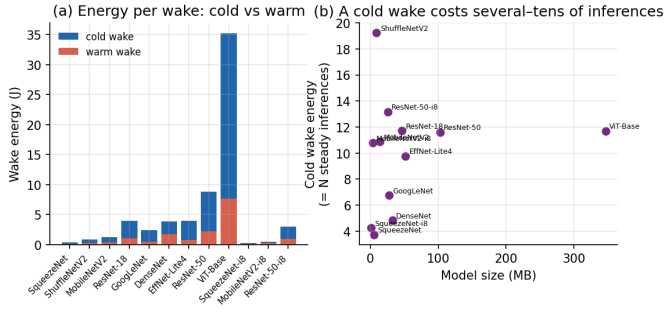


Fig. 7. (a) Cold vs. warm wake energy per model. (b) A cold wake costs the energy of 4–19 steady-state inferences, scaling with model size.

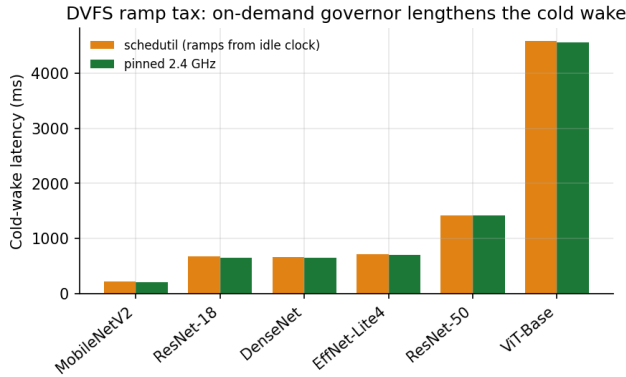


Fig. 8. DVFS ramp tax. Because the wake is I/O-bound, the on-demand governor's clock ramp adds only a few percent to cold-wake latency over a pinned maximum clock.

wake costs from 0.3 J (SqueezeNet-INT8) to 8.8 J (ResNet-50), and 35 J for ViT-Base, scaling with model size; idle draw is 2.1 W. Because loading is I/O-bound the wake draws less instantaneous power than sustained compute, but its duration makes it costly: for a device that wakes to run a single inference, 73–95% of the energy spent is cold-start overhead rather than useful work.

A. The DVFS Ramp Adds Little

Because the cold wake is dominated by flash I/O rather than computation, one might expect the CPU clock to matter little during a wake. Figure 8 confirms it. Comparing the default on-demand governor (`schedutil`), which starts near the minimum clock and ramps up during the wake, against a pinned 2.4 GHz, the governor's ramp adds only 0% to 4% to cold-wake latency (mean $\sim 2.5\%$). The wake is I/O-bound, so paying maximum-clock energy through it is largely wasted—a useful corollary given that max-clock operation is already energetically wasteful for steady-state inference [9].

VII. HOW MANY INFERENCE JUSTIFY A WAKE?

The cold-start tax is a fixed cost per wake, amortized over the B inferences the device runs before sleeping again. Figure 9 plots the cold-start overhead—in both time and energy—as a fraction of the total wake, against B . For a device

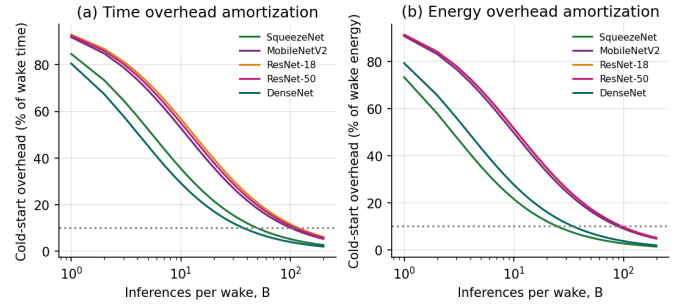


Fig. 9. Amortizing the tax. Cold-start overhead as a fraction of (a) wake time and (b) wake energy versus inferences per wake B . Below $B \approx 38$ –119 the tax dominates.

that wakes to run a single inference ($B = 1$), the overhead is essentially the entire tax: 80–96% of the wake's time and 73–95% of its energy is cold start rather than useful work. Overhead falls below 10% only at $B = 38$ –119 inferences per wake, depending on the model. This quantifies the duty-cycle regime in which cold start matters: a doorbell camera classifying one frame per event lives entirely in the tax; a device batching hundreds of inferences per wake escapes it. The practical consequence is that *the duty-cycle ratio, not the steady-state throughput, determines whether cold start must be engineered around.*

A. Disable Graph Optimization for Short Wakes

ONNX Runtime's default is to apply all graph optimizations, which speeds steady-state inference but lengthens session creation—part of the tax. For a duty-cycled device this default can be counter-productive. Comparing optimization levels, disabling optimization reduces cold-wake latency (a smaller build) at the cost of slower steady inference, so there is a crossover B^* : below B^* inferences per wake, *disabling* optimization minimizes total wake-plus-work time. The crossover is model-dependent: for most models optimization pays for itself after only a few inferences ($B^* < 5$), but for graphs whose optimization is expensive it is substantial— $B^* \approx 18$ for ResNet-18—so a doorbell-style deployment that runs a handful of inferences per wake should disable it. This is a zero-cost configuration change—one enum—that directly reduces the tax for low-duty-cycle deployments, and it composes with the residency policy below.

VIII. GD-TAX: A TAX-AWARE RESIDENCY POLICY

The measurements above imply an operating principle for multi-model edge devices: because a cold wake costs the energy of many inferences and re-triggers whenever memory is pressured, the device should keep *warm* the sessions whose eviction would be most expensive—not simply the most recently or most frequently used. We formalize this as cost-aware caching and evaluate it on the measured constants.

Problem. A device serves a stream of requests to K models but has a RAM budget B too small to hold every session warm. On a request to a resident (warm) model it pays the steady

Algorithm 1 GD-TAX residency on a request to model i

```

1: state: resident set  $R$ , priorities  $H$ , budget  $B$ , aging base  $L$ 
2: if  $i \in R$  then                                     ▷ warm hit
3:    $H_i \leftarrow L + \Delta E_i$            ▷  $\Delta E_i = E_c^i - E_s^i$ , measured
4:   pay ( $L_s^i, E_s^i$ )
5: else                                                 ▷ cold miss
6:   pay ( $L_c^i, E_c^i$ )
7:   while  $\sum_{j \in R} m_j + m_i > B$  do
8:      $v \leftarrow \arg \min_{j \in R} H_j$ ;  $L \leftarrow \max(L, H_v)$ ;  $R \leftarrow R \setminus \{v\}$ 
9:   end while
10:   $R \leftarrow R \cup \{i\}$ ;  $H_i \leftarrow L + \Delta E_i$ 
11: end if

```

cost (L_s^i, E_s^i); on a request to an evicted model it pays the cold wake (L_c^i, E_c^i) and the model becomes resident, evicting others to fit B . Each resident model occupies m_i MB (measured resident footprint, 9–265 MB). The goal is to minimize energy per request (and tail latency) over the trace.

Policy. GD-TAX is a Greedy-Dual policy [10] whose retention priority is keyed on each model’s *measured cold-wake energy penalty* $\Delta E_i = E_c^i - E_s^i$; on access, model i ’s priority is set to $H \leftarrow H_{\min} + \Delta E_i$, and the resident model with the smallest priority is evicted, with the classical aging that raises $H_{\min}$ to the evicted priority. High-tax models are thus retained preferentially, exactly in proportion to what their eviction would cost. Unlike FaasCache’s server keep-alive [5], the cost is a measured on-device energy penalty and the constraint is physical RAM.

Evaluation. We simulate GD-TAX against always-reload (no cache), LRU, LFU, and a cost-aware Belady oracle [24] on the measured constants, over a Zipf popularity workload and a detect→classify pipeline workload, sweeping B from 15% to 100% of the all-resident footprint (Fig. 10). Across both workloads and all budgets, GD-TAX never underperforms LRU or LFU, and it reduces energy per request by up to 15% (Zipf) and 11% (surveillance) at tight budgets, tracking—and at low-to-mid budgets matching or beating—an offline Belady hit-rate reference. The gain is largest where cost and popularity are misaligned (an infrequent but expensive-to-reload detector), exactly the case LRU and LFU mishandle by evicting the costly model between its accesses.

IX. DISCUSSION AND LIMITATIONS

Implications for practice. The results argue for three changes in how edge inference is reported and engineered. (i) *Report the cold wake.* A steady-state latency/energy pair is insufficient for a duty-cycled deployment; a cold-wake latency and energy should accompany it, and the duty-cycle period and memory budget should be evaluation axes. (ii) *Optimize loading, not warm-up.* Because there is no compute warm-up but weight loading dominates, effort is best spent on faster or avoided loading—keeping sessions resident, memory-mapping weights, faster storage, or smaller files—rather than on warm-up loops. (iii) *Budget for re-warm.* On a memory-constrained device the tax recurs whenever a co-tenant’s working set

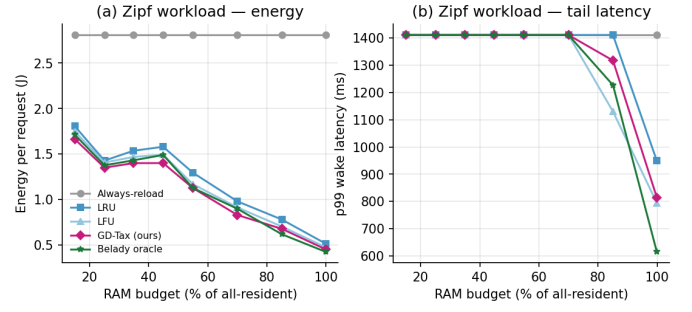


Fig. 10. GD-TAX vs. baselines on measured constants (Zipf workload). By retaining the highest-energy-penalty models, GD-TAX cuts energy per request and tail latency at tight RAM budgets, approaching the Belady oracle and far below LRU/LFU and always-reload.

grows; residency must be managed, which GD-TAX does using the measured energy penalty as its currency.

Generality and threats to validity. Our measurements are ARM Cortex-A76 / ONNX Runtime / microSD specific. The *ratios* we report (tax multiple, disk-dominated decomposition, absent compute warm-up, I/O-bound wake) should transfer to other CPU edge runtimes, but absolute values depend on storage bandwidth (a faster NVMe or eMMC would shrink the disk term and thus the tax) and on the runtime’s graph-optimization cost. The PMIC telemetry samples at ~ 10 Hz, so per-wake energy for the shortest wakes is estimated from pooled power over many repetitions rather than a single window; we report medians and anchor quantitative claims on the well-sampled larger models. The residency policy is evaluated in simulation on measured per-model constants and synthetic-but-realistic traces; an on-device implementation and a field-trace evaluation are natural next steps. Finally, an accelerator (GPU/NPU) execution provider would change the compute term but would, if anything, *increase* the relative weight of loading, strengthening the paper’s thesis.

X. CONCLUSION

Duty-cycled edge inference pays a cold-start tax that steady-state benchmarks make invisible. On a Raspberry Pi 5 the tax is 5–23 $\times$ the steady-state latency, is dominated by loading weights from flash rather than by any compute warm-up, costs the energy of 4–19 steady-state inferences per wake, and re-triggers whenever a small page cache is pressured during idle. These facts change what an edge deployment should optimize: not the hot-loop kernel, but the wake. Our GD-TAX residency policy is one consequence—keeping warm the sessions whose eviction would cost the most energy—but the broader message is methodological. We recommend that edge-inference studies report a cold-wake latency and energy alongside steady-state numbers, and that duty-cycle period and memory budget be treated as first-class evaluation axes.

REFERENCES

- [1] V. J. Reddi, C. Cheng, D. Kanter *et al.*, “Mlperf inference benchmark,” in *ACM/IEEE Int. Symp. on Computer Architecture (ISCA)*, 2020, pp. 446–459.

- [2] S. Bianco, R. Cadene, L. Celona, and P. Napoletano, "Benchmark analysis of representative deep neural network architectures," vol. 6, 2018, pp. 64 270–64 277.
- [3] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *USENIX Annual Technical Conference (ATC)*, 2018, pp. 133–146.
- [4] M. Shahrad, R. Fonseca, I. n. Goiri *et al.*, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *USENIX Annual Technical Conference (ATC)*, 2020, pp. 205–218.
- [5] A. Fuerst and P. Sharma, "Faas-cache: Keeping serverless computing alive with greedy-dual caching," in *Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, pp. 386–400.
- [6] N. D. Lane, S. Bhattacharya, P. Georgiev *et al.*, "DeepX: A software accelerator for low-power deep learning inference on mobile devices," in *ACM/IEEE Int. Conf. on Information Processing in Sensor Networks (IPSN)*, 2016, pp. 1–12.
- [7] M. Xu, J. Liu, Y. Liu, F. X. Lin, Y. Liu, and X. Liu, "A first look at deep learning apps on smartphones," *The World Wide Web Conference (WWW)*, pp. 2125–2136, 2019.
- [8] M. N. Jacob, "The memory wall governs edge DNN inference: A bandwidth-bound characterization of ARM Cortex-A76," (*under review*), 2026.
- [9] —, "Latency-optimal is not energy-optimal: The energy cost of max-clock edge DNN inference," (*under review*), 2026.
- [10] P. Cao and S. Irani, "Cost-aware WWW proxy caching algorithms," in *USENIX Symp. on Internet Technologies and Systems (USITS)*, 1997, pp. 193–206.
- [11] A. Gujarati, R. Karimi, S. Alzayat *et al.*, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 443–462.
- [12] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *USENIX Annual Technical Conference (ATC)*, 2021, pp. 397–411.
- [13] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *Int. Conf. on Learning Representations (ICLR)*, 2016.
- [14] B. Jacob, S. Kligys, B. Chen *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [15] ONNX Runtime developers, "ONNX Runtime," <https://onnxruntime.ai>, 2024.
- [16] F. N. Iandola, S. Han, M. W. Moskewicz *et al.*, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and <0.5mb model size," *arXiv:1602.07360*, 2016.
- [17] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," in *European Conf. on Computer Vision (ECCV)*, 2018, pp. 116–131.
- [18] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [20] C. Szegedy, W. Liu, Y. Jia *et al.*, "Going deeper with convolutions," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [21] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.
- [22] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Int. Conf. on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [23] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Int. Conf. on Learning Representations (ICLR)*, 2021.
- [24] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.